Training quantum neural networks for T1-limited hardware: what the qg symmetry filter does and does not do

Vicente Humberto Monteverde
Universidad del Museo Social Argentino (UMSA), Argentina
ORCID 0000-0001-8884-4811

Draft of 5 October 2026

Abstract

A quantum neural network (QNN) built from gates that conserve the Hamming weight keeps its data in one weight sector, so the shots that left the sector can be discarded at readout. This is the qg filter of the qang library [1, 2]. We show that, with equal amplitude damping (T1) on every qubit, the filtered readout of such a network is exactly the noiseless one in any weight sector. The kept fraction of shots is $(1-\gamma)^{k\,d}$ for weight k and depth d, and training under T1 with the filter gives exactly the parameters of noiseless training. We then measure what this is worth on four small classification datasets. Every result is reported with the filter and without it, and every prediction was committed before its run, failures included. Over 60 runs per model, a network trained on a simulator and run under T1 gains +2.5 points from the filter at weight 1 (95% CI $[0.8, 4.1]$) and +16.5 points at weight 2 ($[11.8, 21.2]$), recovering its noiseless accuracy exactly. A network trained under the calibrated noise without the filter catches up (differences of 0.1 points), and the filter alone corrects neither dephasing nor unequal T1 across qubits, although it loses less than one point up to a $\pm 80\%$ spread of $1/T_1$. Unequal T1 is corrected by training *with* the filter under the calibrated decay rates: the kept state depends only on their ratios, so the model lands within 0.5 points of noiseless at a $\pm 100\%$ spread, ignores a 10% calibration error and, at weight 2, loses 0.7 points under a 1.5-fold drift of all T1 that costs the raw noise-aware model 10. Under equal T1 plus dephasing, the filtered readout equals the dephasing-only one, so zero-noise extrapolation of the filtered features halves (linear) or thirds (Richardson) their error and beats extrapolating the raw readout by 3–10 points, although with 1000 shots per noise scale its variance cancels the gain at weight 2; training with the filter under the calibrated noise recovers the noiseless accuracy under dephasing too. On IBM device noise models, however, where gate errors dominate and the published T1 and T2 give a decay of only 0.002–0.017 per sublayer, training under that calibration added nothing: the filter alone lifted 0.958 to 0.972 (noiseless 0.979), and a calibration measured on the device itself (echo-aware training, or a readout refit on device data) did no better, and neither did adding the published two-qubit gate errors to the training simulator (0.966). Without the filter, the raw multiclass logits are exactly a blend of the noiseless logits with a constant vector fixed by training, which explains why the loss without qang ranged from 3 to 42 points between seeds; a readout linear in the excitation probabilities with no class biases has no such pull, carries the filter inside it and lost nothing noiselessly. With noise-aware training and a richer readout, the model without the filter was 1.2 points better in one study, but over 60 new runs that gap shrank to +0.5 points and was not significant. The weight-2 deficit comes from the pair-product encoding, not from the readout; with an encoding that fills the whole sector and a full-rank readout, weight 2 reaches the weight-1 accuracy. On noise models of IBM and IonQ devices the filter cuts the error of the measured qg_Z by 2.7–3.6×; on the IonQ Forte-1 noise model over 189 inputs of four datasets the filtered model is 1.1 points more accurate than the raw one, and calibrating the errors the filter keeps with five echo circuits per model halves the remaining error of the decision value: decisions that differ from the noiseless model fall from 15 (raw) and 7 (filter) to 3 over 945 readings on five device noise models. With a faithful weight-2 encoding the advantage of the unfiltered model under noise-aware training disappears, and

a classifier given both the filtered and the raw features gains nothing over the filter alone. Over 513 readout configurations, including kept fractions down to 2.6×10^{-7}, unequal T1 and dephasing, the filter has the lower readout error whenever about 20 shots are kept, and a closed-form rule predicts the winner. The same holds beyond the first model: at 8 qubits a weight-2 network keeps its noiseless 0.965 with the filter and drops to 0.705 without it; a five-class classifier (one class per qubit) keeps its noiseless accuracy under T1 with the filter and loses 15–18 points without it on five-class digits; training from 100 or 1000 shots per step with the filter is 1.0–1.5 points ahead of training without it; and the filtered gradient equals the noiseless one, while the raw one is K times smaller at weight 1. On five device noise models the echo calibration turns class confusions back: flipped decisions of the five-class model fall from 51 (raw) and 24 (filter) to 15 over 500 readings. The echo helped only where the filtered error was well above shot noise, and it still removed a third of the filtered error with a 28×28 matrix at 8 qubits. These models are classically simulable and no quantum advantage is claimed. The filter is a way to make noise-free training valid under relaxation, at a shot cost known in advance. The models are released as `qang.qml` (version 0.7.0, `pip install qang`).

1 Introduction

Variational quantum classifiers are trained on simulators and run on noisy devices, or trained directly under device noise. Noise-aware training [3] is the usual remedy when the two differ. Symmetry verification [4, 5] is a different remedy: when the ideal circuit conserves a quantity, outcomes that violate it can be discarded. The qang project expresses qubit readouts in qg units ($qg_Z = \langle\sigma_z\rangle$ and its companions) [1]. It built a register-mean witness and a shot-level filter for circuits that conserve a Hamming weight [2]. This note asks a narrow question: for a QNN that conserves the weight, how much does that filter change the accuracy of a classifier on T1-limited hardware, and when does it stop mattering?

We answer with exact simulation, a fixed protocol and an explicit comparison. Every reading is taken with qang (filtered) and without it (raw), on the same trained parameters or the same initialization. The predictions of every study were committed to the repository before the run that tested them. Of 167 pre-registered predictions in the studies summarized here, 115 passed and 52 failed; the failures are reported alongside the passes.

2 Model and theory

Weight-conserving QNN. On n qubits, a reconfigurable beam splitter (RBS) gate acts on a pair (a, b) as a Givens rotation between $|10\rangle$ and $|01\rangle$ and leaves $|00\rangle$ and $|11\rangle$ fixed, so it conserves the Hamming weight [6]. We use $n = 5$ and three layers of RBS gates on the pairs $\{(0,1),(2,3)\}$, $\{(1,2),(3,4)\}$, $\{(4,0)\}$. That gives 15 angles and depth $d = 9$ (sublayers). The readout is $z = \sum_i c_i\, qg_Z^{(i)} + b$ with trainable c_i, b and a logistic loss.

Encodings. With four features $x \in [-1,1]^4$ let $v = (x,1)/\|(x,1)\|$; the constant component keeps $\|x\|$. Model E (weight 1) puts v_i on the basis state with qubit i excited. Model W (weight 2) puts $v_i v_j$ on the state with qubits $i < j$ excited, normalized. The sectors have dimensions 5 and 10.

The filter. With qang, the readout keeps only the shots whose weight equals the input weight k and renormalizes (`qang.sectors.filter_distribution`). Without qang, all shots are used.

F1/F4: exactness under equal T1. Let amplitude damping with probability γ act on every qubit after every sublayer. Its no-jump Kraus operator multiplies a basis state with k excitations by $(1-\gamma)^{k/2}$: the same factor for every state in the weight-k sector. Every jump lowers the

weight and leaves the sector. The weight-conserving gates never bring a lowered state back. So, conditioned on the weight being k, the state is exactly the noiseless one, and

$$\text{kept fraction} = (1-\gamma)^{k\,d}. \tag{1}$$

For $\gamma = 0.08$ and $d = 9$ this is 0.472 at weight 1 and 0.223 at weight 2. If the damping differs between qubits, the no-jump factor is $\prod_{q\in S}(1-\gamma_q)^{1/2}$ over the excited set S, which is not uniform, and exactness is lost. Dephasing conserves the weight, so the filter cannot see it.

F3: training. Since the filtered T1 readout equals the noiseless one for every parameter value, the loss and all gradients are identical. Training under T1 with the filter therefore yields the parameters of noiseless training from the same initialization.

F5: only the ratios of the decay rates matter. Conditioned on no jump, the state is $A\psi/\|A\psi\|$ with $A = DU_d \cdots DU_1$ and D the diagonal no-jump factor. Multiplying every $(1-\gamma_q)$ by the same constant multiplies D by a constant on the weight-k sector, which cancels in the normalization. The filtered readout therefore depends only on the ratios $(1-\gamma_q)/(1-\gamma_r)$, at any weight (checked to 10^{-12}), whereas the raw readout depends on their absolute values. A model trained with the filter under the device's decay rates needs only the relative T1s.

F6: the filter turns T1 plus dephasing into dephasing alone. Under equal T1, the no-jump factor is a scalar on the sector and commutes with the diagonal dephasing channel, so the filtered readout of a circuit with T1 and dephasing equals that of the same circuit with dephasing alone (checked to 10^{-16}). The filter leaves a single noise parameter, which zero-noise extrapolation can remove.

F7: the raw multiclass readout is a blend. At weight 1 under equal T1 every decayed shot lands in $|0\cdots0\rangle$, where every $qg_Z = +1$ (every $p_c = 0$). The raw logits are therefore $KL_c + (1-K)v_c$, with L the noiseless logits, K the kept fraction and v a constant vector set by training ($W^{\mathsf{T}}\mathbf{1} + b$ for the head readout, b for the qubit readout): without the filter every input is pulled toward the class with the largest v_c. A readout with no class biases and logits linear in the excitation probabilities ($W^{\mathsf{T}}(qg_Z - \mathbf{1})$, or $a\,p_c$) has no pull: the decayed shots, with every $p_c = 0$, only rescale its logits, so its raw decisions equal the filtered ones under any T1, equal or not, and with shots.

Multiclass readout. `MultiClassQNN` uses the weight-1 network with $C \le n$ classes, class c on qubit c. Its "qubit" readout scores class c by the excitation probability $p_c = (1 - qg_Z^{(c)})/2$ with logits $a\,p_c + b_c$; its "head" readout is a linear softmax on the n values $qg_Z^{(i)}$. Since each p_c is a sector probability, F4 applies: under equal T1 the filtered class scores are exactly the noiseless ones. An in-sector error here moves the excitation to another qubit, i.e. swaps one class for another.

F2: simulability. At weight 1 the network is a 5×5 orthogonal matrix O on v, and z is a quadratic form in v. At weight k the sector has dimension $\binom{n}{k}$, polynomial for fixed k. These models can be simulated classically. Whether larger, higher-weight versions could not be is a separate question [7] that this note does not address.

3 Methods

Datasets (binary): iris (versicolor vs virginica), breast cancer, wine (classes 0 vs 1) and digits (3 vs 8), with 100 samples per class where available. Each run uses a stratified 70/30 split,

standardization, PCA to four components where needed, and scaling to $[-1, 1]$; one test sample is worth 2–3 points. Training uses Adam for 120 epochs, with central-difference gradients on the angles. Noise conditions are applied after every sublayer: equal T1 with $\gamma = 0.08$; unequal T1 with γ_q from 0.04 to 0.12 (mean 0.08); and equal T1 plus a phase flip with probability 0.03. The simulation is exact. T1 and dephasing keep the density matrix block-diagonal in the Hamming weight, so one block per weight is propagated; this is checked against the full density matrix to 10^{-12}. Protocol: each study's script, with its predictions written in the header, was committed before the seed of its reported run was used; code was debugged on another seed with few epochs. The commit hashes are in the research notes [1].

4 Results

Single-seed studies. The weight-conserving QNN that keeps the norm reaches a mean accuracy of 0.960 over the four datasets. That is level with a standard (non-conserving) four-qubit QNN (0.952) and with an RBF support-vector machine (0.957) and a small MLP (0.961). Without the norm component the same network gets 0.791. Trained noiselessly and run under T1 with the filter, it keeps exactly its noiseless accuracy. Training under T1 with the filter reproduces noiseless training to 5×10^{-9} in the parameters (F3). At $\gamma = 0.08$, a standard QNN trained noiselessly collapses under T1 (0.771); trained under T1 it recovers (0.944).

The predictions that failed:

- arccos encoding gave no accuracy gain;
- the classical quadratic twin did not match the QNN within 1 point at default regularization;
- unequal T1 cost the filter 1.5 points in one run (bound: 1 point);
- weight 2 was less accurate than weight 1 (by 3.7 points);
- trained under T1, the weight-2 model without the filter was 0.6 points ahead of the filtered one;
- in the studies of Sec. 4: the $\langle Z_i Z_j \rangle$ readout did not help weight 2 (two predictions), the filter was not neutral under noise-aware training with that readout, the gain of the filter at one T1 spread had a CI touching zero, weight 2 was not more sensitive to the spread than weight 1, the pair-product encoding had a lower information ceiling than the unary one, and filling the whole weight-2 sector did not help with the qg_Z readout;
- in Sec. 4: with less than one kept shot the filter did not lose, contrary to the prediction; the raw error of the radius deficit was not smaller at half filling; and three of the four predictions for the two-channel readout, including the replication of the 1.2-point effect above; on device noise models, the filter removed less than half of the false radius deficit of product states; and three of five predictions on distorting encodings, including a second replication of the 1.2-point effect;
- in Sec. 4: all four predictions that the echo would help narrow-margin inputs; training at 1000 shots, where the two readouts were expected within 1 point (qang +1.0), and at 100 shots, where the filter was expected to lose more than a point to its shot cost (it was 1.5 points ahead); the 10-point loss of the multiclass head readout without the filter (9.1 points); a larger loss without the filter at 8 qubits than at 5 at weight 1 (it stayed at 3.1–3.4 points); a raw gradient suppressed more than K at half filling, and fewer shots with the filter in every gradient cell (10 of 12); and, for the multiclass echo, as many agreeing decisions with the echo on every backend and a filtered head readout at least as accurate as the raw one on every backend (each missed on one or two backends by one input); with

3, 5 and 8 classes at 8 qubits, a loss without the filter growing with the classes, at least 15 points with 8 classes, a smaller loss for the qubit readout at every class count (it lost more with 3 classes), and noise-aware training within 3 points (3.0 with 8 classes, at the bound); and, for unequal T1 corrected by calibrated training, a model within 0.5 points of noiseless at weight 2 (0.5 points below) and a larger drift loss for the raw noise-aware model at weight 1 (it lost 0.2 points); and, for dephasing with zero-noise extrapolation, a linear extrapolation removing at least half of the filter's feature error (it left 50.3% and 57.5%), accuracy within 1 point of noiseless at weight 2 (1.3 points below), and no loss against the filter alone with 1000 shots per scale at weight 2 (2.9 points below); and all four predictions that calibrated training from the published T1 and T2 would help on device noise models; all five predictions that a calibration measured on the device would; and a factor of at least 3 between the losses of the top and bottom thirds of the pull index (2.5).

Replication over 60 runs. The key comparisons were repeated over 5 seeds × 3 splits × 4 datasets. Table 1 and Fig. 1 give the results; all five pre-registered predictions of this replication passed.

	E (weight 1)		W (weight 2)	
reading	with qang	without	with qang	without
exact (no noise)	0.952		0.922	
T1, trained without noise	0.952	0.928	0.922	0.757
T1, trained under the noise	0.952	0.951	0.922	0.924
unequal T1, trained without noise	0.947	0.921	0.916	0.732
dephasing, trained without noise	0.918	0.792	0.882	0.656
dephasing, trained under the noise	0.950		0.926	

Table 1: Mean test accuracy over 60 runs per model, with and without the qg filter.

The paired differences (mean, 95% CI across seeds) were as follows.

- *Gain from the filter, trained without noise, under T1:* +2.5 $[0.8, 4.1]$ points (E) and +16.5 $[11.8, 21.2]$ (W). The filtered model was ahead or tied in 50/60 (E) and 56/60 (W) runs.
- *With noise-aware training:* +0.1 $[-0.6, 0.9]$ (E) and −0.1 $[-0.8, 0.5]$ (W).
- *Loss from unequal T1:* 0.5 $[-0.2, 1.2]$ (E) and 0.7 $[0.0, 1.3]$ (W). The 1.5 points of the earlier run were at the top of the spread.
- *Dephasing, noise-aware minus clean training with the filter:* +3.2 (E) and +4.5 (W), both CIs above 0.
- *Under dephasing, a model trained without noise still gains* +12.6 (E) and +22.6 (W) points from the filter, which removes the T1 part of the combined noise.
- *Weight 2 against weight 1:* −3.0 $[-4.0, -2.0]$ points.

Readout, encoding and T1 spread (60 runs each).

- *Correlation readout.* Adding the $\binom{5}{2} = 10$ correlations $\langle Z_i Z_j \rangle$ makes the weight-2 readout a linear function of the whole sector distribution (rank 10 instead of 5). It did not help: −0.6 $[-1.5, 0.4]$ points. Trained under T1 with 15 readout weights, the model *without* the filter was 1.2 $[0.4, 2.0]$ points better. The decayed shots still carry information about where the excitations were, and the filter discards it. The filter is still exact for this readout, and trained without noise it adds +14.9 points.

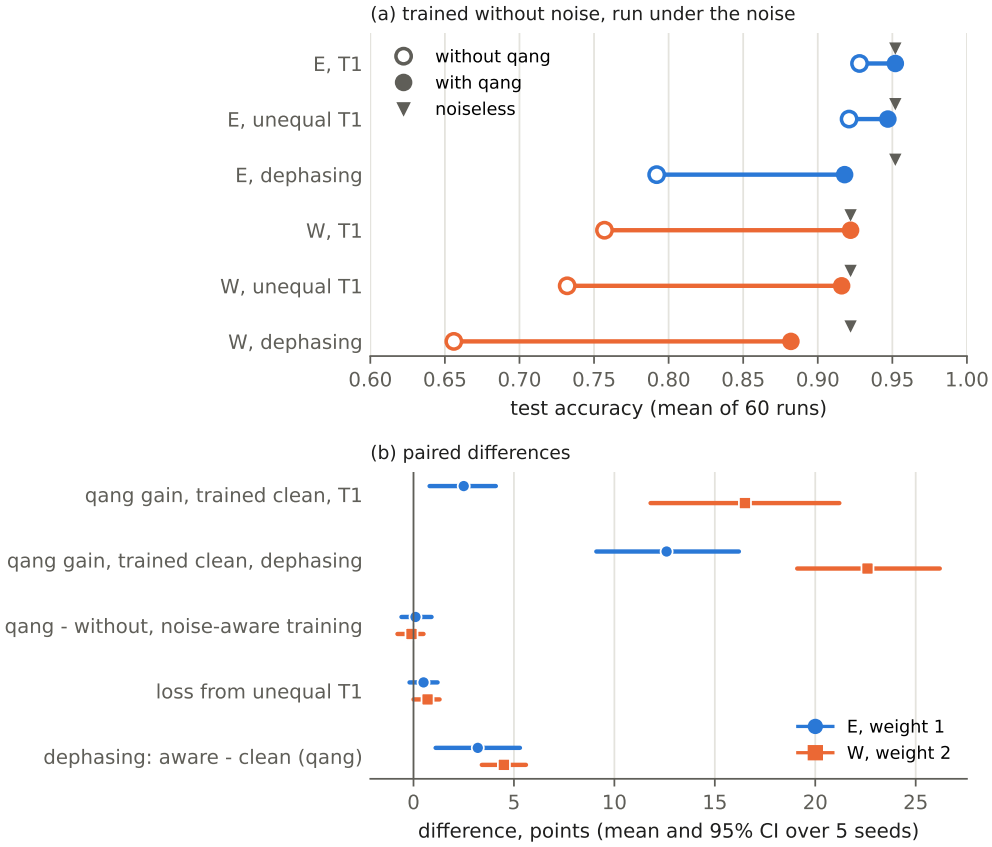


Figure 1: (a) Accuracy of networks trained without noise and run under the noise, without (open) and with (filled) the filter; triangles mark the noiseless accuracy. (b) Paired differences, mean and 95% CI across 5 seeds.

- *Encoding.* The best unconstrained quadratic form in the pair-product amplitudes reaches 0.928, 2.4 $[1.7, 3.0]$ points below the same ceiling for the unary amplitudes (0.952). The normalization of the products distorts the data. Loading the weight-1 data on five weight-2 states instead (a "ring" of pairs $(k, k+1)$) gives 0.943, within 1.0 point of weight 1 and 2.3 $[0.7, 3.9]$ points above the pairs. With this encoding the gain of the filter under T1 is the largest seen, +23.9 points. A "dual" encoding fills all ten weight-2 states without distortion (v on the ring, $(x^2, 1)/\|(x^2, 1)\|$ on the chords $(k, k+2)$). With the qg_Z readout it does not help (-0.2 points against the ring), but with the full-rank $\langle Z_i Z_j \rangle$ readout the weight-2 model reaches the weight-1 accuracy (0.947 against 0.947 over 60 runs). Weight 2 needed both a faithful encoding and a readout that sees the whole sector.

- *T1 spread.* With $\gamma_q = 0.08(1 + s\,u_q)$ and u_q spread over $[-1, 1]$, the loss of the filter was 0.3/0.2 points (weight 1/2) at $s = 0.5$ and 1.4/1.3 at $s = 1$. The filter alone keeps the loss below one point up to about $s \approx 0.8$, i.e. a $\pm 80\%$ spread of the decay rates $1/T_1$ around their mean.

Device noise models. The weight-1 model was compiled to Qiskit (a cascade of 4 RBS gates loads the signed unary amplitudes; 15 trained RBS gates follow; the circuit matches the model to 10^{-16}). It was run on 30 iris test inputs with 1000 shots each, read with and without the filter from the same shots. The noise models were those of three IBM devices (calibration-based fake backends simulated by Aer) and the IonQ cloud simulator with the Aria-1 and Forte-1 noise models. On all five the accuracy was 0.933 with and without the filter, equal to the noiseless

accuracy: the margins of this model are wide. The error of the measured qg_Z fell from 0.138–0.207 without the filter to 0.040–0.069 with it (2.9–3.6×), keeping 69–74% of the shots. On the IonQ Forte-1 noise model the same pipeline was run on all four datasets (189 test inputs). The pooled accuracy was 0.968 with the filter, 0.958 without and 0.979 noiseless, and the qg_Z error was 2.7–2.9 times smaller with the filter on every dataset. (IonQ results use the measured frequencies returned by the API; qiskit-ionq's `get_counts` resamples them on each call.) These are noise models, not devices; the hardware runs are prepared, with cost confirmations, and pending.

Echo calibration of the errors the filter keeps. The filter discards shots that left the sector, but errors that move an excitation inside it pass: on the device noise models 8–23% of the kept shots of a product state put the excitation on the wrong qubit. An echo circuit (qubit j excited, the 15 trained RBS gates and their inverse) should return qubit j; its filtered distribution is column j of a 5×5 transfer matrix M, and inverting $M^{1/2}$ on the filtered distribution of each input mitigates those errors, as readout mitigation does for bit flips (`qang.sectors.echo_transfer_matrix`, `unmix_sector`). With 5 echo circuits per trained model, on the 189 inputs of the four datasets, the three IBM noise models and the IonQ simulator in its native gate set (IonQ's compiler removes identities from circuits in its standard gate set, so echoes must be sent natively):

- the error of the decision value against the noiseless model fell by 43–56% relative to the filter alone, and filter plus echo was 3.7–4.6 times below the raw readout;
- over 945 readings, 15 decisions differed from the noiseless model without the filter, 7 with it and 3 with the filter plus the echo calibration; on the three IBM noise models filter plus echo reached the noiseless accuracy (0.979);
- all twelve predictions of these three studies passed.

Two follow-up studies set the limits of the echo. In the weight-2 sector (a 10×10 matrix from 10 echoes, 30 inputs, the same five backends) it removed 37–43% of the qg_Z error left by the filter and 39–48% of the $\langle Z_i Z_j \rangle$ error, and all four predictions passed. On 120 narrow-margin inputs of a weight-1 model, chosen before the noisy runs, all four predictions failed: the filter cut flipped decisions from 47 to 17 over 600 readings, and the echo left 19. There the filtered error (0.26–0.35) was close to the shot-noise scatter of the decision value (about 0.22), so little bias was left to remove and inverting the matrix added variance. The echo is worth its circuits when the filtered error is well above its shot-noise level, which a noiseless simulation at the same shots shows. At 8 qubits and weight 2 (28 sector states, 28 echoes at 4000 shots, the same five backends) the sector is dirtier: only 50–80% of an echo's kept shots return to the prepared state, against 70–90% at 5 qubits. The echo still removed 27–38% of the qg_Z error left by the filter (32% on average, against 40% at 5 qubits) and filter plus echo was 3.4–4.1 times below raw; all five predictions passed.

Two channels, shot cost and what else the filter reads.

- *Two-channel readout (60 runs).* Giving the classifier both the filtered and the raw features (2×15 weights, `qang="both"`) was meant to keep the information of the decayed shots that made the raw model 1.2 points better above. With the dual encoding that effect did not replicate: trained under T1, the filtered model equals the noiseless one (0.953) and the raw one reaches 0.950 (-0.3 $[-0.8, 0.2]$ points against the filter). The two channels equal the raw readout (-0.0 $[-0.6, 0.6]$) and, at weight 1, the best single readout ($+0.0$ $[-0.4, 0.4]$).
- *Distorting against faithful encoding (60 new runs).* Trained under T1 with the pair-product encoding and the 15-weight readout, the raw model was ahead by $+0.5$ $[-0.7, 1.6]$ points,

not 1.2; with the dual encoding it was behind by -0.4 $[-1.2, 0.4]$, and the difference between the two ($+0.9$ $[-0.6, 2.4]$) was not significant. With the 5-weight qg_Z readout the filter was better by 0.6 $[0.1, 1.0]$ points. The one case where the filter lost does not hold up as a significant effect.

- *Shot cost.* For the qg_Z readout of random weight-conserving circuits the filtered error follows $(1 - z^2)/(KS)$ and the raw error $(r - z)^2 + (1 - r^2)/S$, with K the kept fraction and S the shots. Over 108 configurations (4–8 qubits, weight 1–4, depth 3–24) the filter has the lower error in 106 (up to 1000× lower), and the rule picks the winner in 99%; the break-even is 3–110 shots. Over 405 further configurations with K down to 2.6×10^{-7}, unequal T1 and dephasing, it wins in every configuration with at least 20 kept shots (up to 1871× under equal T1, 94× unequal, 58× with dephasing), and a generalized rule with the biased filtered value picks the winner in 342 of 342 cases with $KS \geq 5$. When fewer shots survive, the raw readout has also lost the information, so the filter still did not lose.

- *Entanglement after the filter.* In a fixed-weight state each qubit's Bloch radius is $|qg_Z|$, and for a pure register $1 - qg_Z^2$ is its entanglement with the rest. Under equal T1 the filtered value equals the noiseless one to 3×10^{-15}; on product states it is exactly 0, while the raw readout shows a false value of 0.60–0.95. On five device noise models (IBM and IonQ) the filter cuts the error of this readout 3.2–4.0 times on trained circuits, but errors that move an excitation inside the sector pass the filter and still make product states look entangled.

Beyond the first model: shots, classes, size and gradients. Table 2 collects the studies that each change one thing in the setting above.

- *Training from shots* (4 datasets × 5 seeds). Model E was trained under T1 from S fresh shots per evaluation (SPSA on the angles) and tested at 1000 shots. With the filter it reached 0.939 ($S = 1000$) and 0.930 ($S = 100$), without it 0.929 and 0.915. The shot cost of the filter did not show in training. Noiseless training plus the filter at run time (0.945) was as good as any route.

- *Multiclass* (iris, wine, digits 0–4; 5 seeds). With the filter every run equals its noiseless accuracy. Without it the head readout loses 9.1 points on average and 18 on five-class digits; the qubit readout loses less, since under equal T1 the raw p_c shrink together (on three-class iris it was even 0.9 points ahead). Trained under the noise, the raw model stays within 2 points.

- *5, 6 and 8 qubits* (cancer, wine, digits; 3 seeds; depth 9). The RBS layers are built directly in each weight sector (`block_unitaries`), which makes 8-qubit training 13 times faster. With the filter every run equals its noiseless accuracy. Without it the weight-1 loss stays at 3.1–3.4 points from 5 to 8 qubits; at weight 2 it grows from 8.7 to 26.0 points. Noise-aware training without the filter stays within 0.9 points in every cell.

- *Gradients* (random circuits, $n = 4, 6, 8$, weight 1 and $n/2$, 200 draws per cell, $\gamma = 0.02$). The filtered gradient equals the noiseless one; the raw gradient variance is K^2 times smaller at weight 1, and less suppressed than K at half filling, where decayed shots keep part of the signal. The sector itself flattens the gradient, with or without the filter, by 5.2× (weight 1) and 20× (half filling) from 4 to 8 qubits. Counting kept shots, the filter resolved the gradient with fewer shots in 10 of 12 cells; at $n = 8$, $k = 4$ ($K \leq 0.14$) it needed 1.3–2.2 times more.

- *Multiclass on device noise models* (digits 0–4, 100 inputs, 1000 shots, the five backends of Sec. 4, 5 echo circuits per model). Here an in-sector error is a class confusion, and the echo corrects part of it: averaged over the backends the head readout gave 0.858 raw, 0.890 with the filter and 0.900 with filter plus echo, against 0.91 noiseless.

- *Eight classes on eight qubits* (digits $0..C-1$, $C = 3, 5, 8$; 5 seeds). With the filter every run equals its noiseless accuracy. Without it the head readout loses 0.7, 15.9 and 11.1 points for 3, 5 and 8 classes, with a large spread between seeds (2–34 points with 5 classes); the qubit readout loses 5.1, 4.4 and 3.4. Unlike at 5 qubits, noise-aware training without the filter stays 2–3 points behind with 5 and 8 classes.

- *Unequal T1 corrected* (iris, cancer, wine, digits; 5 seeds; γ_q from 0 to 0.16). Trained with the filter under rates calibrated with a 10% error, the model reached 0.951 (weight 1) and 0.940 (weight 2) against 0.950 and 0.945 noiseless; the filter alone gave 0.931 and 0.918, the raw readout 0.873 and 0.797. Trained without the filter under the same rates it reached 0.952 and 0.941. When all T1 shortened by 1.5 after training, the filtered model lost 0.8 and 0.7 points and the raw one 0.2 and 10.0 (F5).

- *Dephasing with zero-noise extrapolation* (same data and seeds; T1 0.08 plus dephasing $p = 0.03$ or 0.06; noise scales 1, 2, 3 as by gate folding; features extrapolated linearly or by Richardson). At $p = 0.03$ the filter alone gave 0.933 / 0.919 (weight 1 / 2), filter plus linear ZNE 0.949 / 0.943 and Richardson 0.953 / 0.946, against 0.952 / 0.956 noiseless; ZNE of the raw readout gave 0.915 / 0.844. At $p = 0.06$, weight 2, filter plus ZNE gained 10.6 points over the filter alone. With 1000 shots per scale, filter plus ZNE gave 0.945 / 0.864 against 0.932 / 0.893 for the filter alone, and Richardson fell to 0.710 at weight 2. Training with the filter under the calibrated noise reached 0.951–0.956 at both dephasing levels (F6).

- *The whole pipeline on device noise models* (the IBM fake backends of Sec. 4, the 189 inputs of the four datasets, a fixed layout). The published T1 and T2 of the chosen qubits and the circuit duration gave a decay of 0.002–0.017 and a dephasing of 0.001–0.005 per sublayer. Trained noiselessly, the model gave 0.958 raw and 0.972 with the filter (with or without the echo), against 0.979 noiseless; trained with the filter under that calibration it gave 0.968, and trained without the filter under it 0.968. Gate errors, which a T1/T2 calibration does not describe, dominate these noise models; the echo cut the decision error but changed no decision.

- *Calibrating with the device itself* (same backends and inputs). Retraining with the filter through the in-sector mixing measured by the echo gave 0.970, and refitting the linear readout on 40 training inputs run on the device gave 0.965, against 0.972 for noiseless training plus the filter.

- *Why the multiclass loss varies between seeds* (5-class digits, 20 seeds per readout). F7 predicted every raw decision in all 40 runs. A pull index, $(1 - K)/K$ times the spread of v over the median training margin, ranked the seeds by their loss without qang (Spearman 0.98 for the qubit readout, 0.65 for the head); the losses ran from 3 to 42 points, and with qang every run was noiseless.

- *Dephasing at 8 qubits* (cancer, wine, digits; 3 seeds; T1 0.08 plus dephasing 0.03). As at 5 qubits, linear filter plus ZNE left 50% and 56% of the filter's feature error and reached 0.955 / 0.962 (noiseless 0.959 / 0.964) with exact probabilities; with 1000 shots per scale at weight 2 it fell to 0.879, below the filter alone (0.915); training with the filter under the noise gave 0.963 / 0.961. All five predictions passed.

- *A pull-free multiclass readout* (5-class digits, 10 seeds). Dropping the class biases cost nothing noiselessly (0.938 against 0.932 for the head readout). Without the filter, the pull-free head readout kept 0.938 under equal T1 and 0.928 under unequal T1, against 0.653 and 0.624 for the standard readout, and its raw decisions equalled the filtered ones in every run and condition. All five predictions passed.

- *Gate errors in the training simulator* (the IBM backends and inputs above; a two-qubit depolarizing error after every RBS gate, $p = 0.015$–0.028, from the published ECR or CZ error). Trained with the filter under T1/T2 plus that gate error the model gave 0.966, against 0.972 for noiseless training plus the filter; trained without the filter under it, 0.970, 1.2 points above the noiseless model read raw. The simulator overestimated the kept fraction by 0.03–0.12, plausibly because it has no readout error. One of five predictions passed.

study	setting	noiseless	with qang	without qang
shots (T1)	trained at $S = 1000$ / $S = 100$	0.945	0.939 / 0.930	0.929 / 0.915
multiclass (T1)	qubit / head readout	0.918 / 0.953	0.918 / 0.953	0.865 / 0.863
size (T1)	8 qubits, weight 1 / weight 2	0.977 / 0.965	0.977 / 0.965	0.946 / 0.705
gradients (T1)	variance ratio, weight 1	1	1	K^2
multiclass (devices)	head, flips of 500	0.91	0.890 (24)	0.858 (51)
	with echo calibration		0.900 (15)	
multiclass, 8 qubits (T1)	head, 5 / 8 classes	0.952 / 0.916	0.952 / 0.916	0.793 / 0.805
unequal T1 ($\pm$100%)	weight 1 / 2, filter alone	0.950 / 0.945	0.931 / 0.918	0.873 / 0.797
	trained with calibrated rates		0.951 / 0.940	0.952 / 0.941
	after a 1.5-fold T1 drift		0.942 / 0.932	0.950 / 0.841
T1 + dephasing 0.03	weight 1 / 2, filter alone	0.952 / 0.956	0.933 / 0.919	0.815 / 0.686
	with linear ZNE (exact)		0.949 / 0.943	0.915 / 0.844
	with linear ZNE (1000 shots)		0.945 / 0.864	0.909 / 0.841
	trained with filter under noise		0.953 / 0.956	
IBM noise (189)	weight 1, clean training	0.979	0.972	0.958
	T1/T2-calibrated training		0.968	0.968
	device calibration		0.970 / 0.965	
	gate-error-aware training		0.966	0.970
T1 + deph., 8 qubits	weight 1 / 2, filter alone	0.959 / 0.964	0.925 / 0.950	0.788 / 0.650
	with linear ZNE (exact)		0.955 / 0.962	0.914 / 0.851
multiclass head, T1	standard / pull-free	0.932 / 0.938	0.932 / 0.938	0.653 / 0.938

Table 2: Studies beyond the first model, with and without the filter (mean accuracy unless stated).

5 Discussion

The difference the filter makes depends on how the network was trained. Trained on a simulator and run under relaxation, the filter is decisive. Its effect grows with the weight, because a weight-k state decays k times faster. Trained under a calibrated noise model, a network without the filter learns the decay, and the filter adds nothing measurable. What the filter provides is therefore procedural rather than an accuracy gain over the best alternative. Under equal T1 it makes noise-free training exactly valid, with no noise model and no noisy training runs, and its cost (the discarded shots) is known before running. Unequal T1 is corrected when the network is trained with the filter under the calibrated decay rates: since only their ratios enter (F5), this needs the relative T1s, tolerates calibration errors and, at weight 2, survives a common drift that the raw noise-aware model does not. Dephasing is corrected the same way, by training with the filter under the calibrated noise; and because the filter turns T1 plus dephasing into dephasing alone (F6), zero-noise extrapolation of the filtered readout removes most of the remaining bias of observables, at a shot cost that at 1000 shots per scale cancelled the gain at weight 2. The filter and calibrated training are most useful together where decay or dephasing is strong; on device noise models dominated by gate errors, a calibration from the published T1 and T2, or with the published gate errors added, gave nothing, and noiseless training plus the filter was the

best pipeline. Two practical rules follow from the studies above. Up to about a ±80% spread of $1/T_1$ across qubits, a network trained on a simulator can be run with the filter alone, losing less than one point. When the network is instead trained under the calibrated noise with a distorting encoding and a readout with many parameters, the decayed shots looked useful in one study (1.2 points), but over 60 new runs the gap was 0.5 points and not significant; with a faithful encoding it was absent, and adding the raw features to the filtered ones gained nothing. For weight-2 models, how the data are loaded matters more than what is read out. The shot cost of the filter did not decide against it anywhere in our scaling maps. What the filter cannot see, errors that keep the excitation in the sector but move it, can be calibrated with echo circuits at the cost of five extra circuits per model, provided the filtered error is well above shot noise. These conclusions held when the model was trained from shots, extended to five classes, or grown to 8 qubits; with size, the loss without the filter grew only at weight 2. The filter also hands training the noiseless gradient, but it does not cure the flattening that the sector itself brings at larger n.

Limitations. All noise is simulated, including the device noise models. There is one architecture per weight, 5–8 qubits, and small datasets in which one test sample is worth 2–3 points. The weight-2 model reaches but does not exceed the weight-1 accuracy on these datasets. No quantum advantage is claimed: these networks are classically simulable. A hardware run with the filter is the natural next test: a guarded IonQ package for the five-class model with echo (55 circuits, 60 000 shots) is ready, rehearsed on the Forte-1 noise model (8, 4 and 1 decisions differing from the noiseless model raw, with the filter and with the echo, over 50 inputs), with its predictions committed, and waits for hardware credits.

Code and data availability

Models: `qang.qml` in `qang` 0.7.0 (`pip install qang`), NumPy only, with `WeightQNN` (weights 1 and 2, readouts `"z"` and `"zz"`, encodings `"pairs"`, `"ring"` and `"dual"`; any weight for simulation; `qang="both"`), `kept_fraction` and `compare_qang`; `MultiClassQNN` and `block_unitaries`; echo calibration in `qang.sectors` (`echo_transfer_matrix`, `unmix_sector`). Hardware-ready script: `examples/qnn_hardware_qg.py` (IBM and IonQ). Studies: `examples/qnn_*_qg.py` and RESEARCH_NOTES §75–§82, §85, §87, §90, §91 and §94–§101, §104–§124, with tests in `tests/`. The Colab notebooks `notebooks/qang_qml.ipynb` (English) and `qang_qml_es.ipynb` (Spanish) install the library from PyPI and show every result with and without the filter. Repository: https://github.com/Viny2030/qang.

```
from qang.qml import WeightQNN
m = WeightQNN(n_qubits=5, weight=2).fit(X_train, y_train)
m.score(X_test, y_test, gamma=0.08, qang=True) # = noiseless accuracy
m.score(X_test, y_test, gamma=0.08, qang=False) # raw readout
```

References

[1] Vicente Humberto Monteverde. The Qang (qg): A unified angular-probability unit and metric for parametric quantum circuit design. 2026. doi: 10.5281/zenodo.22832150.

[2] Vicente Humberto Monteverde. The register-mean polar bias as a symmetry witness: error mitigation, constrained optimization and qubit characterization in qg units. *Zenodo preprint*, 2026. doi: 10.5281/zenodo.23036963.

[3] Hanrui Wang, Jiaqi Gu, Yongshan Ding, Zirui Li, Frederic T. Chong, David Z. Pan, and Song Han. QuantumNAT: quantum noise-aware training with noise injection, quantization

and normalization. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2022. doi: 10.1145/3489517.3530400.

[4] Xavier Bonet-Monroig, Ramiro Sagastizabal, M. Singh, and Thomas E. O'Brien. Low-cost error mitigation by symmetry verification. *Physical Review A*, 98:062339, 2018.

[5] Sam McArdle, Xiao Yuan, and Simon Benjamin. Error-mitigated digital quantum simulation. *Physical Review Letters*, 122:180501, 2019.

[6] Jonas Landman, Natansh Mathur, Yun Yvonna Li, Martin Strahm, Skander Kazdaghli, Anupam Prakash, and Iordanis Kerenidis. Quantum methods for neural networks and application to medical image classification. *Quantum*, 6:881, 2022. doi: 10.22331/q-2022-12-22-881.

[7] M. Cerezo et al. Does provable absence of barren plateaus imply classical simulability? Or, why we need to rethink variational quantum computing, 2023. arXiv:2312.09121.